Hummingbird+: Advancing FPGA-based LLM Deployment from Research Prototype to Edge Product

Jindong Li
Institute of Automation, Chinese Academy of Sciences
Beijing, China

Tenglong Li
Institute of Automation, Chinese Academy of Sciences
Beijing, China

Guobin Shen
Institute of Automation, Chinese Academy of Sciences
Beijing, China

Dongcheng Zhao
Institute of Automation, Chinese Academy of Sciences
Beijing, China

Qian Zhang
Institute of Automation, Chinese Academy of Sciences
Beijing, China

Yi Zeng
Institute of Automation, Chinese Academy of Sciences
Center for Long-term Artificial Intelligence
Institute of AI Safety and Governance
Beijing, China

Abstract

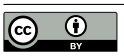
Field-Programmable Gate Arrays (FPGAs) have been shown to be viable for Large Language Model (LLM) deployment, but they remain less competitive than embedded GPUs and NPUs for final edge products. This is largely because existing FPGA-based LLM accelerator prototypes rely on large, expensive FPGA devices to provide sufficient hardware resources for satisfactory performance, whereas edge products are highly cost-sensitive. In this work, we move beyond pure architectural prototyping to evaluate the feasibility of using low-cost FPGAs as the final implementation medium for LLM deployment. We propose ***Hummingbird+***, which encompasses: (1) a compact embedded FPGA-based LLM accelerator designed to deliver comparable inference performance compared to embedded GPUs and NPUs, and (2) a custom Printed Circuit Board (PCB) built around a Zynq UltraScale XCZU2CG/3EG SoC, equipped with 24GB of memory and an expected Bill of Materials (BOMs) under $150 in mass production. Through extensive FPGA-centric optimizations, we significantly reduce the accelerator's resource consumption, enabling deployment on entry-level FPGAs with exceptional cost efficiency. On this platform, we successfully deploy the GPTQ 4-bit Qwen3-30B-A3B LLM, achieving a decoding speed of over 18 token/s and a prefill speed of over 50 token/s without further model compression. To our knowledge, this is the first demonstration of an FPGA-based edge product serving as a practical and cost-effective final implementation medium for LLM deployment.

CCS Concepts

• **Hardware → Hardware accelerators**.

Keywords

Large Language Model, Edge Deployment



FPGA 2026, Seaside, CA, USA.

ACM ISBN 979-8-4007-2079-6/2026/02
https://doi.org/10.1145/3748173.3779189

ACM Reference Format:
Jindong Li, Tenglong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2026. Hummingbird+: Advancing FPGA-based LLM Deployment from Research Prototype to Edge Product. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA 2026), February 22–24, 2026, Seaside, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3748173.3779189

1 Introduction

In the era of deep learning, Field-Programmable Gate Arrays (FPGAs) play several critical roles. On one hand, their reconfigurability allows researchers to explore novel architectures[18, 55, 64] without incurring the high cost of chip tape-out, democratizing integrated circuit design and customized computing[10]. On the other hand, FPGAs have long served as indispensable verification platforms[42] for Application-Specific Integrated Circuits (ASICs). These two applications have significantly accelerated the development of AI processors. The third role of FPGAs is serving as the final deployment platform. Successful examples include BrainWave project[11] and Vitis AI[27]. However, as deep learning models grow in complexity, GPUs continue to advance rapidly[51], and ASICs rise in prominence[26], FPGAs—viewed purely as computational devices—struggle to match the TFLOPs/TOPs of these alternatives, becoming increasingly less competitive in such role.

Recently, however, Large Language Model (LLM) has begun to reshape this landscape[7, 36, 45]. LLMs exhibit memory-bound nature, reducing the decisive impact of raw compute throughput on end-to-end inference performance. Hardware platforms with vastly different computational capabilities are thereby brought closer to the same starting line, as memory bandwidth—often comparable across embedded devices—emerges as the primary performance bottleneck. In this context, ***the disadvantage of FPGAs in peak compute density becomes less pronounced.***

Yet, sustaining LLM inference still requires moving billions of parameters across memory for every generation step, a burden that overwhelms the limited bandwidth of embedded devices. A critical breakthrough addressing this challenge is the Mixture-of-Experts (MoE) LLM[25][37], where each token activates only a small subset of experts, dramatically reducing bandwidth requirements, enabling

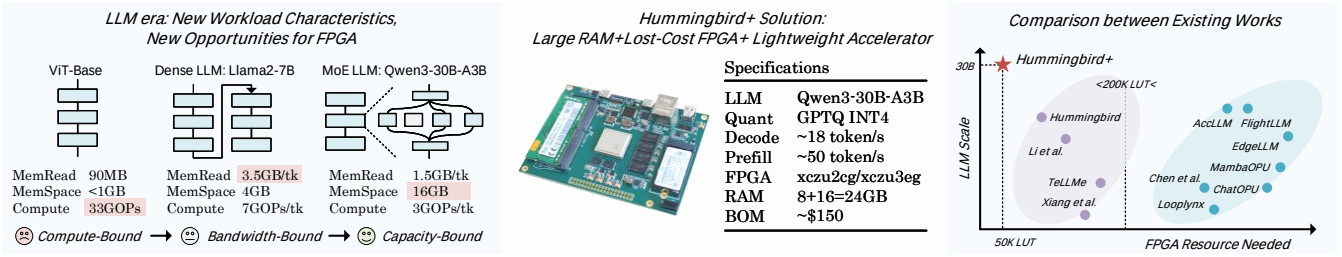


Figure 1: Our proposed Hummingbird+ solution for MoE-based LLM deployment.

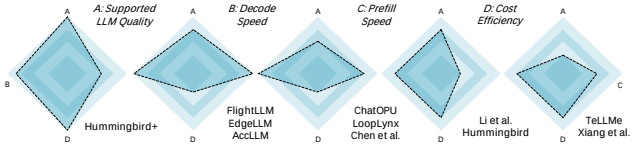


Figure 2: Radar chart of advantages of various accelerators.

the use of slower but higher-capacity memory. Here, embedded FPGAs reveal a unique advantage: although typically limited to DDR4, ***their flexible I/O allow interfacing with multi-channel memory, thereby compensating for single-channel speed limitations while supporting larger aggregate capacity.*** This synergy between MoE's nature and FPGA's scalable memory connectivity reopens the possibility of FPGAs serving not only as prototypes playground or verification platforms, but as final edge-deployment products for MoE LLMs—an opportunity that CNNs[19], ViT[13], BERT[12], and dense LLMs[52] could not provide.

Despite emerging opportunities, most prior work still focuses on architectural prototyping with large, high-end FPGAs rather than deployable solutions, and MoE LLM deployment is yet to be explored. FlightLLM[62] and AccLLM[35] ran LLaMA2-7B on the Alveo U280, while Chen et al.[8] implemented GPT-2-1.5B on the same platform. EdgeLLM[23] deployed ChatGLM-6B on the VCU128, another HBM-equipped FPGA. LoopLynx[67] deployed GPT-2-345M on the U50, ChatOPU[66] ran OPT-1.3B on the U200, and LightMamba[57] and MambaOPU[39] deployed Mamba2-2.7B on the VCK190 and U200. The high cost of these FPGAs hinders pratical productization. Recent work on embedded FPGAs aims to lower this barrier: Li et al.[31] deployed LLaMA2-7B on the KV260, and Hummingbird[29] further cut resource use to enable even cheaper platforms. However, both focus only on decode-stage acceleration. TeLLMe[43] introduced prefill acceleration on the KV260 but targets a much smaller BitNet LLM and relies on extreme ternary quantization, which severely limits LLM quality. Xiang et al.[59] achieved high prefill speed but target 0.6B LLM. ***The hardware cost, LLM quality, prefill and decode speed appear to form an impossible trade-off***, where improving any one dimension typically comes at the expense of the others, as shown in Fig.2.

Recognizing these existing limitations, we propose ***Hummingbird+***, which aims to simultaneously achieve the following goals: (1) Support a high-quality, medium-scale LLM without radical compression method. (2) Target a low-cost FPGA chip. (3) Approach the theoretical decoding speed limit. (3) Attain as much prefill acceleration as possible. Our objective is to advance FPGA-based LLM deployment from research prototypes toward final edge products. The accelerator-level innovations are summarized as follows.

- We introduce a series of optimizations to the *GEMV engine*. Building on techniques such as integer packing, double data rate, cascading, prefetching and chain-tree mixing implementation, we further enhance performance through: **A.** eliminating the double data rate LUT multiplexer, **B.** enabling dual-precision computation (W4 for linear projection and KV8 for attention), and **C.** supporting two GEMV modes to avoid costly transpositions (DOT for linear projection and AXPY for attention). Our single engine achieves 272 GOPs using 140 DSPs at 532MHz with ***just <1K LUTs overhead.***
- To handle floating-point nonlinear operations such as softmax and RMSNorm, we implement significant resource-saving techniques in the *scalar engine*. Specifically, **A.** packing and double data rate are applied to the 16-bit floating-point multiplier, halving DSP usage, and **B.** detailed analysis of data temporal dependencies and resource occupancy enables effective operator-level resource sharing. As a result, we reduce the scalar engine footprint to ***<6K LUTs and only 7 DSPs***, supporting all of the LLM special functions while still fully hidden within the MatMul cycles.
- The optimized GEMV and scalar engines together form a ***token processor*** capable of decode acceleration. Leveraging the resource savings achieved above, we manage to instantiate 2 and 4 processors on XCZU2CG and XCZU3EG, further enabling prefill acceleration with DSP utilization of 64% and 85% (and near 80% and 100% CLB utilization respectively).

Most importantly, we have taken the first step toward productization by designing a custom printed circuit board (PCB) (Fig.1) with specifications tailored to support medium-scale MoE LLMs. Our design adopts entry-level Zynq Ultrascale XCZU2CG and XCZU3EG with 24GB memory integrated on-board, providing sufficient capacity to accommodate the GPTQ-INT4[15] Qwen3-30B-A3B[60] LLM, which offers significantly better performance than 7B dense LLM[53]. On XCZU3EG, our accelerator achieves over 18 token/s decoding speed and 50 token/s prefill speed. The entire PCB can be produced at a bill of materials (BOMs) of $150 in mass production, making it a cost-effective solution to deploy a 30B LLM on edge.

2 Preliminaries

2.1 Mixture of Expert LLM

The Mixture-of-Experts (MoE) design expands the capacity of large language models by replacing a conventional feed-forward layer with a group of parallel expert modules. Instead of processing every token through a single dense network, MoE introduces E independent experts $\{f_1, \ldots, f_E\}$ and a routing mechanism that decides

Table 1: Configuration parameters, adopted notations and actual value of the Qwen3-30B-A3B LLM

Config.	Note.	Val.	Config.	Note.	Val.
Hidden State Dim	d	2048	Total Experts	E	128
Expert Dim	d_e	768	Activated Experts	E_a	8
Head Dim	d_h	128	Layer	L	48
Query Head	H_q	32	Vocab Size	v	151936
Key/Value Head	H_{kv}	4	Context Length	T	256K

which few experts should handle each token. For a token $\mathbf{x} \in \mathbb{R}^d$, the router first applies a linear projection followed by a softmax:

$$\mathbf{p} = \text{softmax}(W_r \mathbf{x}), \tag{1}$$

$$\tilde{p}_i = \begin{cases} \dfrac{p_i}{\sum_{j \in \text{TopK}(\mathbf{p})} p_j} & \text{if } i \in \text{TopK}(\mathbf{p}), \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

where $W_r \in \mathbb{R}^{E \times d}$ are learnable routing parameters, and $\mathbf{p} \in \mathbb{R}^E$ contains the activation scores for all experts. The router then selects the top-K scored experts and renormalizes their probabilities. Each chosen expert f_i is implemented as the same three-layer transformation commonly used in LLM feed-forward blocks: a gating, up and down projection:

$$f_i(\mathbf{x}) = W_d^{(i)} \Big[(W_u^{(i)} \mathbf{x}) \odot \sigma(W_g^{(i)} \mathbf{x}) \Big], \tag{3}$$

$$\mathbf{y} = \sum_{i \in \text{TopK}(\mathbf{p})} \tilde{p}_i \, f_i(\mathbf{x}). \tag{4}$$

where $W_g^{(i)}, W_u^{(i)}, W_d^{(i)}$ are the gating, up, and down projection of the i-th expert, and $\sigma(\cdot)$ is a non-linear function, and the MoE layer output y is the weighted combination of the selected experts.

By activating only a small subset $K \ll E$ for each token, LLM achieves a much larger effective parameter without increasing the memory access cost proportionally. In recent developments, the sparsity of MoE-based LLMs have continued to increase, evolving from the 8-sel-1 of Mixtral-8x7B[25], to 64-sel-8 in DeepSeek-V2-Lite[37], to 128-sel-8 in Qwen3-30B-A3B[60], and even to 512-sel-11 in Qwen3-Next-80B-A3B[44]. As the sparsity ratio of MoE LLMs increases, the memory access workload per token approaches the sweet spot for low-cost DDR-based hardware, making embedded FPGAs an increasingly viable deployment solution.

2.2 Resource and Performance Modeling

Here, we model the memory capacity requirements of the MoE-based LLM and estimate the expected prefill and decode performance based on the available compute and memory bandwidth. Without loss of generality, we adopt the Qwen3-30B-A3B model as a representative template for MoE-based LLMs, assuming symmetric GPTQ INT4 quantization[15] with group size = 128.

2.2.1 RAM Space Requirments: The pre-post attention normalization and q, k normalization[63][21] per layer require $2(d + d_h)$ parameters in 16-bit. Attention projections including query, key, value, and output contain $2dd_h(H_q + H_{kv})$ parameters. The router consumes dE parameters. Each expert, comprising the up, gate, and down projections, uses $3dd_e$ parameters. The final LM head projection adds dv parameters, and d 16-bit normalization scale. All these projection layers employ INT4 quantization with a 16-bit scale per group, equivalent to an average of 4.125 bits per parameter. The kv cache at context length T requires $2Td_hH_{kv}$ elements, stored with INT8 quantization and a 16-bit scale per d_h group, giving an effective 8.125 bits per cached value. Therefore, the total RAM space required to host a typical MoE-LLM parameter is:

$$4.125dv + 16d + L\Big[32(d + d_h) + 4.125\big[2dd_h(H_q + H_{kv}) + dE + 3Edd_e\big]\Big] \text{ bits}$$

which corresponds to a total of **14.52GB** of parameters for Qwen3-30B-A3B, with only **1.47GB** of parameters activated per token. Note that we apply the embedding-table offloading technique [29] to offload the embedding-table to external storage and free additional RAM space. The additional RAM space required for a 16K-context kv cache is:

$$8.125 \times 2LTd_hH_{kv} \text{ bits}$$

Although Qwen3-30B-A3B can natively support a context length of up to 256K, we choose to support up to 16K due to implementation constraints. Under this setting, **0.77GB** of RAM is required.

2.2.2 Decoding and Prefilling Speed Modeling: For multi-head attention (MHA)-based LLMs[54], the decoding process is entirely memory-bound, meaning that the decoding speed depends solely on memory bandwidth. In contrast, for group-query attention (GQA)[4], where the kv cache is shared among heads, compute capacity also affects decoding speed as the context length increases. Let the compute-to-bandwidth ratio be denoted as α, which equals one for existing decoding accelerators[42][31][29]— a value that also matches the number of *token processors* used in this work. Under these conditions, the theoretical decoding speed and prefilling speed, measured by the *Time per Output Token* (TPOT) and *Time to First Token* (TTFT) metric[2], is given by:

$$TPOT_{\text{Ideal}}(t) = \frac{B}{M_{\text{ActivatedParam}} + H_q/H_{kv}/\alpha \cdot M_{\text{KVCache}}(t)} \text{ token/s.}$$

$$TTFT_{\text{Ideal}}(t) = \frac{T \cdot M_{\text{ActivatedParam}}}{\alpha \cdot B} + \frac{T \cdot H_q/H_{kv} \cdot M_{\text{KVCache}}(t)}{2\alpha \cdot B} \text{ s.}$$

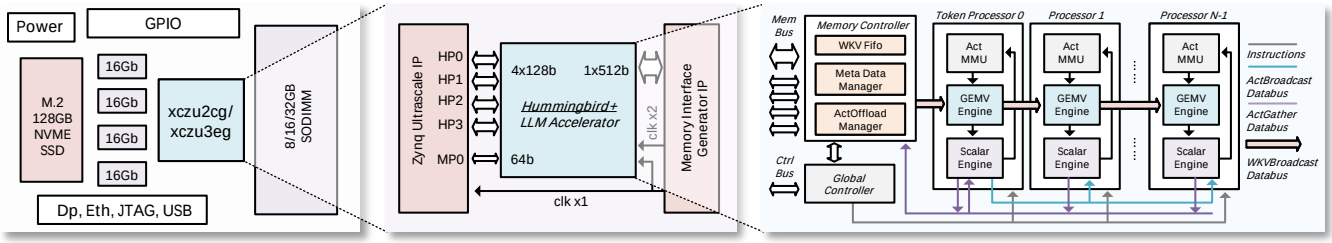
We will measure the actual performance to evaluate how closely it approaches the theoretical roofline in the experiment section.

3 Top-Down Overview of Hummingbird+

In this section, we present a top-down overview shown in Fig.3, starting from the customized PCB platform, to the system-on-chip (SoC) organization, and finally down to the accelerator architecture.

3.1 Platform Level: PCB Overview

Our cunstomized PCB platform, shown in Fig.3A, adopts entry-level Zynq Ultrascale SoC, XCZU2CG or XCZU3EG, which share compatible pin package. The processor system (PS) side integrates 8GB DDR4 on-board memory, while the programmable logic (PL) side features a SODIMM slot supporting 8/16/32GB of memory. The dual-channel memory interface delivers a peak bandwidth of 34GB/s, which is comparable to the throughput of modern embedded NPU ASICs[46] equipped with DDR5, but offers significantly larger capacity (24GB versus the typical 8GB). For non-volatile storage, an M.2 NVMe slot is reserved for a 128GB SSD, leveraging the built-in PCIe-2.0 hard core of the PS to enable fast model-weight loading and historical kv cache retrieval during initialization. These on-board resources can sustain inference for 30B MoE-based LLMs.



(A) Platform Level: PCB Overview (B) SoC Level: FPGA System (B) Accelerator Level: Architecture Organization

Figure 3: Top-Down overview from platform level, SoC level, downto accelerator level.

3.2 SoC Level: FPGA System

The on-chip organization of system components, shown in Fig.3B, follows the block design diagram in Vivado Design Suite. The Zynq Ultrascale processor provides four 128-bit slave AXI HP ports for memory access to the PS-side DDR4, and an additional master AXI port serving two purposes: (1) issuing instruction control to the accelerator, and (2) initializing PL-side memory, such as during weight loading. On the PL side, the Memory Interface Generator (MIG) IP exposes a 512-bit slave AXI port for access to PL-side DDR4 and supplies two phase-aligned clocks—one at 266MHz and one at 532MHz (double data rate)—to the accelerator. Our block design resembles the architecture in [29], but with key optimizations to reduce logic consumption. We eliminate the AXI interconnect IP between AXI HP ports and the accelerator (saving around 1K LUTs per 128-bit channel), remove the AXI crossbar bridging the Zynq master AXI and the accelerator-to-MIG connection (saving around 3K LUTs), and omit the AXI Datamover IP, which otherwise consumes significant BRAM. Instead, we implement a custom AXI infrastructure in RTL to achieve higher resource efficiency.

3.3 Accelerator Level: Arch. Organization

Existing LLM accelerators typically follow a three-pillar design—a matrix engine, a nonlinear special-function unit, and a memory manager[62][35][66][24]. We identify several inefficiencies: (1) LLM inference is dominated by *flat-shaped* GEMM[22] operations (reducing to GEMV during decoding). A generic matrix engine with mismatched array sizes can underutilize compute and memory resources. (2) Decoding is highly sensitive to data dependencies and initialization latency. A coarse-grained three-block design lacks the fine-grained pipeline needed to hide idle cycles. (3) Loose coupling between the matrix and special-function units forces the latter to be over-provisioned for extra parallelism to prevent nonlinear OPs from stalling the MatMul pipeline, inflating resource usage.

To overcome these limitations, we propose a novel accelerator organization centered on a highly resource-efficient and compact ***token processor***. Each token processor integrates all the necessary components (a GEMV instance for MatMul, a scalar engine for special functions, and an activation buffer for on-chip decode dataflow[62]) to execute a complete single-token inference (decode-like), taking inspirations from existing decode accelerator designs[31][29]. For prefill acceleration, we instantiate multiple token processors, shown in Fig.3C, sharing the memory bus used to access model weights and kv cache. Because computations for different prefill tokens have minimal interaction, this architecture avoids complex interconnections between processors while enabling scalable multi-token inference. ***In the prefill stage***, all token processors actively participate in both the attention and MLP layers. ***In the decode stage***, however, the first token processor assumes the role of the main processor: it handles the full computation while the remaining processors skip MLP operations but collaboratively share the GQA workload. Identical instructions are issued to all processors, with a flag mechanism indicating whether a processor should execute or discard a given command during the decode stage. Lightweight interconnection (16-bit data bus) links between the main processor and other processors enable activation broadcasting and gathering. We argue that such organization is efficient in ***low-batch LLM inference***[28][3] in edge scenarios.

4 Inference Dataflow

This section presents the fine-grained dataflow during the decode and prefill stages of LLM inference. In the attention layer, we highlight the prevalent GQA design in recent LLMs shown in Fig.4, while in the MLP layer, we focus on the MoE dataflow.

4.1 Decode Stage Dataflow

GQA has become the dominant design in recent LLMs. Dataflows optimized for MHA [31] are unsuitable because the $q : k : v = 1 : 1 : 1$ ratio no longer holds. Existing GQA dataflows [29] lack group-level parallelism and require large on-chip buffers to cache all kv pairs. We propose a simpler, more efficient GQA dataflow that enhances efficiency and enables parallel execution across multiple token processors. As shown in Fig. 4A, k and v are computed in advance since they are shared across multiple q vectors. With 4 token processors, we first compute q_0, q_1, q_2, q_3 sequentially, apply normalization and RoPE[49] in pipeline, and forward q_1, q_2, q_3 to their respective processors. All processors then perform the qk DOT product, softmax, and $softmax \cdot V$ operations in parallel. An online softmax is implemented to reduce the required iterations from three to two. To hide the tail latency of RoPE and softmax, the next set of k and v projections is split into chunks and scheduled between q projection, qk DOT, and $softmax \cdot V$. For the final attention group, where no remaining k or v projections can be split and scheduled, the output o projection is used instead to fill the tail-latency gap. The MoE dataflow during the decode stage is straightforward, only the main token processor is involved in computation. An expert-by-expert execution strategy is sufficient.

4.2 Prefill Stage Dataflow

Iterative Chunk Attention: In the attention layer, the accelerator performs a T-token chunked attention, where T denotes the number of token processors. All attention outputs are first offloaded

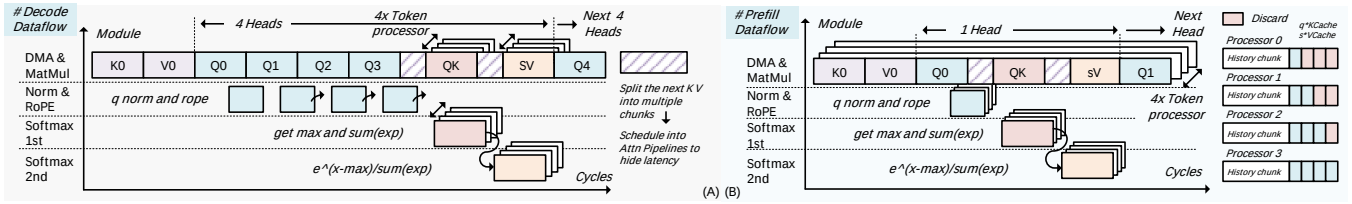


Figure 4: Attention dataflow during prefill and decode stage.

to external memory, and the system iterates over all prefill tokens before proceeding to the MoE computation. Unlike the decode stage—where token processors collaboratively share the GQA workload of the same token—each token processor in the prefill stage *handles its own token* independently, with no cross-processor interaction. The q vectors remain local, requiring no forwarding. Each processor executes its own head-wise q projection, qk dot product, and $softmax \cdot V$ pipelines, with scheduled k, v, and o projections inserted between them, shown in Fig.4B. The only computation difference between processors is the casual mask. Consider the case of $T = 4$, where four processors with IDs $0, 1, 2, 3$ compute tokens $t_k, t_{k+1}, t_{k+2}, t_{k+3}$, respectively. After each processor computes the key and value of its assigned token and offloads them to external memory, the subsequent computations of $q \cdot K_{\text{Cache}}$ and $softmax \cdot V_{\text{Cache}}$ must follow the causal mask. Specifically, processor ID = 0 discards the results for $t_{k+1}, t_{k+2}, t_{k+3}$; processor ID = 1 discards t_{k+2}, t_{k+3}; and processor ID = 2 discards t_{k+3}, ensuring that each query only attends to valid past tokens.

Token-wise Top-K Scores Calculation: After the chunked attention computation, we observe that the router logits can be calculated immediately, allowing the Top-K score calculation to be fully overlapped with the ongoing process—effectively hiding their latency before the MoE computation begins. This stage is fused into each chunk-attention iteration to maintain a continuous pipeline. The selected experts and their normalized softmax scores are cached on-chip to simplify instruction scheduling and address generation.

Expert-wise MoE Calculation: Unlike the token-wise computation in previous stages, the MoE computation is carried out expert-wise. For each expert, we repeat the following steps: fetch the RMSNorm results of the attention outputs for the tokens that have selected that expert, compute the up, gate, and down projections, and update the corresponding partial sums. This process continues until all tokens assigned to the current expert have been processed, after which the computation proceeds to the next expert.

5 Microarchitecture Optimizations

In this section, we provide optimization details of the GEMV and the scalar engine, the main components of a single token processor. Existing research has shown that FPGA resources are generally sufficient if supporting decoding only[31][29]. However, practical usage demands a low TTFT latency, which in turn requires prefill acceleration, causing resource consumption to scale linearly, making it challenging to fit within the limited resources of embedded FPGAs. Nevertheless, ***FPGA resources are like a water-soaked sponge—if carefully optimized, additional capacity can always be squeezed out.*** We present our best practices for aggressively reducing the resource footprint of a single processor, enabling the instantiation of multiple processors to accelerate the prefill stage.

FPGA designers possess a widely adopted *arsenal* of DSP-oriented optimizations for building efficient compute engines, such as operand packing[48][65], Double Data Rate (DDR)[58][33], Single Instruction Multiple Data (SIMD)[32][34], and DSP cascading[30][47]. However, assembling these techniques into a combination still demands meticulous, fine-grained manual design. We summarize all our optimizations in Table.2, along with the corresponding DSP built-in circuits. The table includes references (1)-(9) that are highlighted and explained in the following sections. In this work, the term DSP specifically refers to the ***DSP48E2*** in Ultrascale FPGA.

5.1 GEMV Engine

Our proposed GEMV engine (Fig.5A) supports dual-precision —W4 for linear projection and KV8 for attention—and operates in two modes: the DOT mode ($y = W \cdot A$) and the AXPY mode ($Y_i = a \cdot X + Y_{i-1}$). The engine achieves a parallelism of M=2, K=128, N=2. Here, M=2 arises from the DDR technique, while N=2 is enabled by operand packing in W4 precision (downgrading to N=1 in KV8 precision). The K=128 parallelism is realized through the parallel instantiation of DSPs. ***The GEMV engine consumes only <1K LUTs and 140 DSPs through the following aggressive optimizations.***

Dual Precision Operand Packing: We adopt a classic W4KV8 quantization scheme for LLM inference. For activations, the common empirical choice is 16-bit; however, to better match the accumulation bit-width of the DSPs, we reduce it to 12-bit with support for dynamic quantization and dequantization. In our design, the 12-bit activation occupies the B input port, while the weights and kv cache share the A and D input ports, leveraging the DSP's pre-adder to perform a classic $(A+D) \times B$ packing (1). As demonstrated in Fig.5C, in W4 mode, the 8-bit data bus feeding the DSP carries a pair of 4-bit weights. One is placed in the least significant bits of D, and the other in the most significant bits of A after signed extension. In KV8 mode, the A input port is gated to zero via the DSP's internal INMODE[1] signal (2), while the D input port retains its least significant 4 bits and concatenates the remaining 4-bit KV data to form the full 8-bit input. Leveraging the DSP's internal logic to implement the zero-gating of the A port reduces LUT consumption, requiring only five 2-to-1 multiplexers per 8-bit data bus to handle precision-mode switching at the D port. It is also worth noting that data packing of signed integers requires a +1 correction term, which is conventionally implemented using additional LUTs[16]. We observe that this correction can instead be absorbed into the DSP's quad-input post-adder by configuring and selecting the constant rounding factor (6) at the W multiplexer, thereby eliminating the need for extra LUT-based correction logic.

Chain-Tree Mixing Implementation: Through empirical analysis of quantization group size, typical LLM head dimensions, and

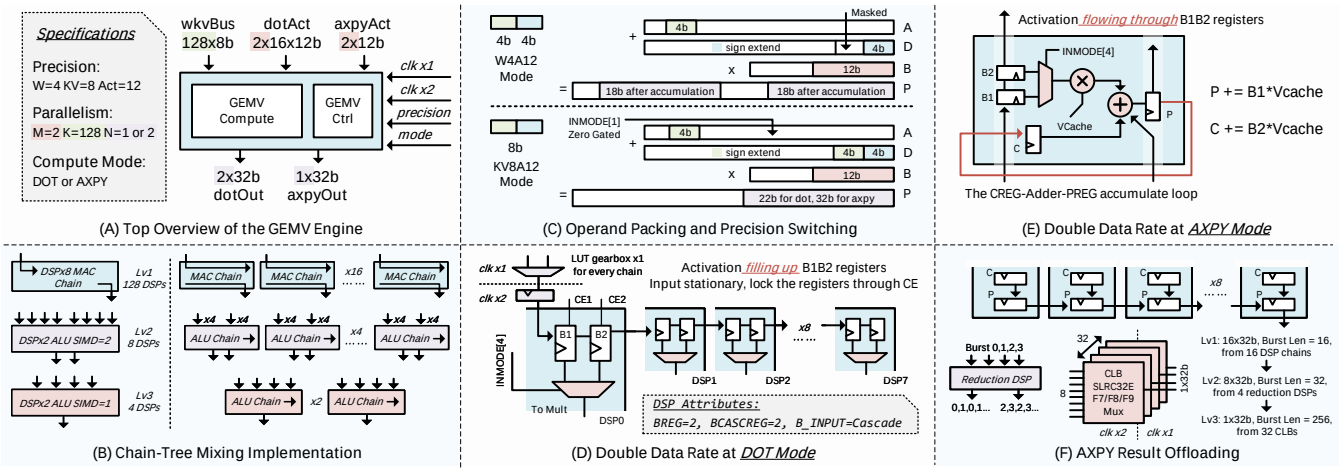


Figure 5: Implementation and optimization details of the GEMV Engine.

Table 2: DSP built-in resources and roles in optimization

#	DSP Built-in Resources	Roles in Optimization
①	A+D Preadder	Operand Packing/Data MUXing
②	INMODE[1][2] Gate	Precision Switching/Data MUXing
③	INMODE[4] Mux	Data MUXing
④	B Cascade Path	Operand Prefetching/Broadcasting
⑤	B1B2 Register CE	Input Stationary Data Locking
⑥	RND Factor at WMUX	Result Correction
⑦	C and P Register	Ring Accumulation at AXPY Mode
⑧	P Cascade Path	Accumulation/Offloading
⑨	SIMD Add	Multi-channel Reduction

the memory bus width of the FPGA chip, we determine a GEMV parallelism of 128 as the optimal hyperparameter. To balance DSP utilization, systolic FF usage, and place-and-route efficiency, we group the DSPs in sets of 8, forming 16 DSP chains with both input-B cascade ④ and output-P cascade ⑧ paths connected in chain. Each of these 128 DSPs performs either two 4×12 signed multiplications or a single 8×12 signed multiplication. The 12-bit activation is constrained to be symmetric, allowing each DSP chain to produce either two 18-bit outputs or a single 22-bit output after accumulation. To aggregate the 16 partial sums generated by the chains, we implement a hierarchical reduction tree. At the first stage, a 4-to-1 reduction unit—comprising two cascaded DSPs operating in ALU mode—performs the initial summation. Four such units carry out the 16-to-4 reduction, with the DSPs operating in SIMD=2 mode ⑨, delivering a pair of 24-bit reductions for the 4×12 precision mode or a single lane for the 8×12 mode. Since the intermediate 24-bit width is insufficient for the 8×12 output, a second stage employs two SIMD=1 4-to-1 reduction units for the final aggregation. In total, six reduction units are used—four in SIMD=2 mode and two in SIMD=1 mode—consuming 12 DSPs exclusively for reduction. Consequently, a single GEMV engine requires 128 DSPs for MACs plus 12 DSPs for reduction, for a total of 140 DSPs, shown in Fig.5B.

Double Data Rate in DOT Mode: We apply the DDR technique only on the activation side, rather than on the weights or *kv* cache, to avoid entangling it with operand-packing optimizations and complicating the design. Conventional DDR implementations—such as those used in AMD Vitis AI DPU[14]—require additional LUTs to function as a *gearbox*, which switches between two data segments from the slow clock domain to provide one segment to the fast domain. However, our design aims to minimize LUT usage, making it impractical to instantiate a separate LUT gearbox for every DSP. Moreover, adding a 12-bit activation bus for every DSP would consume excessive routing resources. To address these constraints, we adopt an input-stationary dataflow combined with an in-DSP prefetching technique [30]. Activations are loaded into the DSPs through the B cascade chain ④, starting from the bottom of the chain. By leveraging the dual pipeline registers on the B input path, each DSP can store two 12-bit activations, while the DDR switching is handled internally by the DSP's multiplexer, controlled via the INMODE[4] signal ③. Loading a DSP chain requires only 8 slow-clock cycles (or 16 fast-clock cycles), but because activations remain stationary for a long duration—depending on the MatMul size—this loading overhead is negligible to performance. After the all the B pipeline registers in the DSP chain is loaded, the clock enable signals of the B1 and B2 register are cleared to *lock* the activation ⑤. In this way, only one LUT-based gearbox per DSP chain is required, placed at the start of the DSP chain shown in Fig.5D.

Double Data Rate in AXPY Mode: On the input side, unlike the $y = W \cdot A$ DOT mode, where each DSP consumes a different activation a, the AXPY mode ($Y_i = a \cdot X + Y_{i-1}$)[56] uses a shared activation a across all DSPs. As a result, instead of ***filling up*** the B-registers with activation—where they would remain locked after loading—the activation ***flows through*** the B-register pipeline, broadcasting the same value along the systolic path with the B1/B2 clock enable signals always set ⑤. The DDR technique on the activation side still applies: by controlling the INMODE[4] signal ③, two sets of AXPY factors can be interleaved at the fast-clock domain. On the output side, unlike DOT mode, where partial sums

Table 3: Comparison of different GEMV engine designs

Impl.	Prec.	MACs	AXPY	LUT	FF	DSP
[31]	W16A16	128	No	31871	44809	256
[29]	W4A24	128	Yes	1962	4355	148
Ours	W4KV8A12	512/256	Yes	950(+912)	2313(+2231)	140

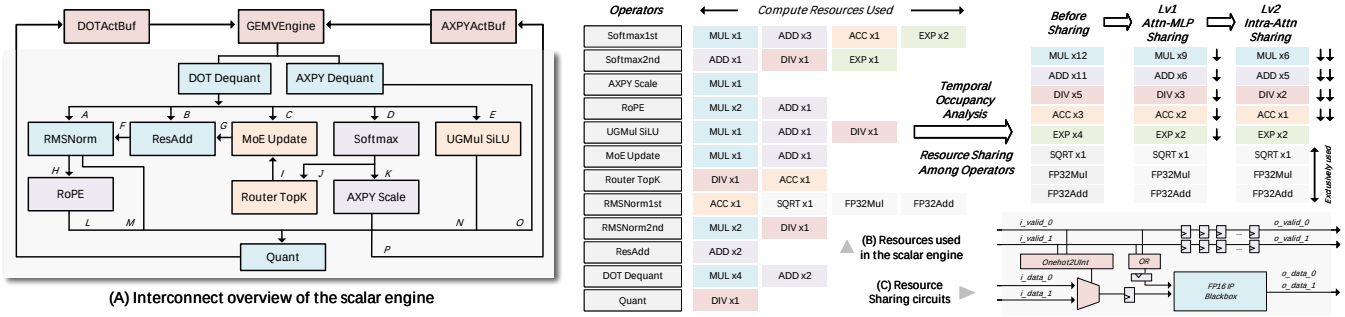


Figure 6: Overview of the scalar engine and the resource sharing optimizations.

are propagated along the chain, AXPY mode requires in-place accumulation of results. However, each DSP contains only a single P register, while two different AXPY results must be stored and updated alternately. Therefore, we exploit the unused C register by routing the P output back to the C input port, forming a CREG-adder-PREG ring (7) for accumulation, shown in Fig.5E. This loop enables alternating updates of the two AXPY results entirely within the DSP. As a result, DDR accumulation in AXPY mode is achieved with only extra routing resources with no LUTs/FFs overhead.

AXPY Result Offloading: Unlike DOT mode, where partial sums are accumulated along the P cascade path and flow out through the top DSP element of the chain, AXPY mode produces two independent results per DSP that cannot be merged across the chain. A straightforward solution is to directly fetch outputs from every DSP's P port, but this would create prohibitively high fanout. Moreover, the parallel outputs of the AXPY results must be serialized before being sent to the scalar engine. To avoid excessive LUT-based multiplexing for serialization, we adopt a three-level serialization as shown in Fig.5F. We first reuse the P cascade path for data offloading (8). By disabling the post-adder (setting the W, X, and Y multiplexers to zero) and enabling the Z multiplexer to select only the cascaded P input, we form a clean offloading path that shifts the AXPY results upward through the chain, one element at a time, to the top DSP. This produces a burst of 16 elements per chain (with 16 chains in total) once AXPY accumulation completes. However, caching such a wide-bus, shallow-burst output is still inefficient. To further serialize, we repurpose the first-level reduction DSPs to convert the 16 sets of length=16 bursts into 8 sets of length=32 bursts. We then employ the SRLC32E shift-register primitive, which first stores the offloaded AXPY results from the reduction DSP, and then uses the CLB F7/F8/F9 multiplexers to perform serialization to finally produce a fully serialized stream of 256 AXPY outputs for subsequent computation. These components are entirely within a single CLB with minimum net or logic delay, making it fully capable of operating at doubled clock rate. Such approach requires only 32 CLBs in total, and the entire offloading process takes just 16 slow-clock cycles, negligible to overall runtime. Table.3 compares existing GEMV engine designs. We achieve the highest MAC density, 2× at KV8 precision and 4× at W4 precision, while using the fewest DSPs. The additional LUT and FF usage in the bracket (*wkv process* module listed in Fig.9) comes from precision switching and systolic flip-flops, which is shared across multiple GEMV engines.

5.2 Scalar Engine

As shown in Fig.6, our proposed scalar engine receives the DOT or AXPY outputs from the GEMV engine, converts the fixed-point results to FP16 (dequantization), performs a suite of special functions and then converts the FP16 outputs back to fixed-point integers (dynamic quantization) before sending the data back to the activation buffer for next MatMul. There are two main approaches to implementing the scalar engine. The ***overlay approach***, which builds a fully featured functional unit that integrates all necessary computational resources (multipliers, adders, exponential units) and provides an instruction set to orchestrate these resources for operations (e.g., softmax or RMSNorm)[39][20]. However, instruction-driven computation disrupts the pipeline dataflow, and a one-size-fits-all unit can only guarantee functional support rather than efficient execution. The ***dataflow approach***, more common in FPGA-based architectures[61][9], where dedicated pipeline logic is designed for each specific operator (e.g., softmax or RMSNorm). To prioritize efficiency, we adopt the dataflow approach. The function of each submodules in the scalar engine and the interconnections of the submodules labeled (A) to (P) are explained below.

The RMSNorm module receives either the q or k projection outputs (A) to perform qk normalization (then forwarding the results (H) to the RoPE module) or the residual outputs (F) to perform attention or post-attention normalization. The ResAdd module receives the o attention projection outputs (B) or the final MoE outputs (G) to perform residual addition. The MoE-Update module receives the down-projection outputs (C) and expert scores (I) from the Router module and updates the MoE results expert by expert in an iterative manner. The Softmax module receives either the qk DOT outputs or the router projection outputs (D), and returns the MoE scores (J) or the attention scores (K) required for the subsequent $softmax \cdot V$ Axpy operation. The UGMul-SiLu module fuses element-wise multiplication and the SiLU activation function between the up and gate projection outputs (E). The outputs of RMSNorm (M), RoPE (L), UGMul (N), and the AXPY results (O) are then passed through the dynamic quantization unit to convert FP16 activations to FIX12 activations, which are subsequently sent to the activation buffer for DOT operations. The outputs of the AXPY-Scale module (P), which multiplies the attention scores by the scaling factor of the v cache, are sent to the activation buffer for the AXPY operation. It is worth noting that the actual hardware implementation may differ and be more complex than the simplified illustration here.

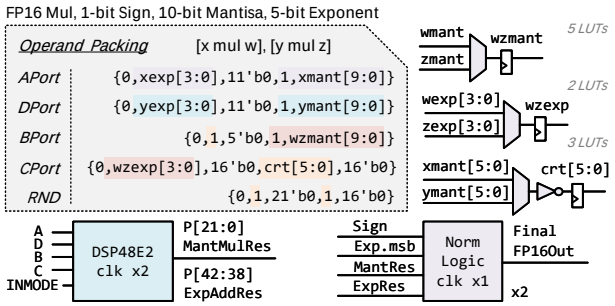


Figure 7: Optimizations of floating point multiplier.

However, a dataflow-based scalar engine typically requires more resources, as each submodule demands independent compute units. Fig.6B lists the FP16 compute resources for each submodule (the online softmax[41] and the RMSNorm are split into two stages), suggesting 12 MUL, 11 ADD, 5 DIV, 3 ACC, 4 EXP, 1 SQRT, as well as 2 FP32 MUL and 1 ADD IP resources are needed. A naive implementation exceeds the resource budget of an embedded FPGA. ***We manage to reduce the scalar engine footprint to <6K LUTs and only 7 DSPs through the following aggressive optimizations.***

Resource Sharing via Occupancy Analysis: We break the internal barriers of the scalar engine by: (1) exposing the required compute I/O externally instead of instantiating IPs within each submodule; (2) assembling all submodules at the top level to analyze compute I/O traffic and temporal occupancy; and (3) identifying non-overlapping compute I/O operations and applying the resource-sharing circuits in Fig.6C to reuse the same FP16 IP resources across multiple compute I/O ports. The sharing circuit in Fig.6C uses the valid signals of all ports as a one-hot vector. A onehot-to-index converter selects the payload of the active port and forwards it to the FP16 IP. The output valid signal is simply a delayed version of the input valid signal, matching the internal compute latency of the IP, and the IP output is broadcast to all ports.

To ensure correctness, only one valid signal may be asserted at any given time, hence temporal occupancy analysis of compute I/O ports is necessary. We conduct a two level analysis. ***First***, we route temporally non-overlapping compute ports to the same resources. Operators active only during the attention stage naturally do not conflict with those used in the MLP stage. For example, EXP and DIV in UgMulSiLU can be shared with Softmax, and the ADD used in MoEUpdate can be shared with RoPE. ***Second***, we address potential conflicts among RMSNorm, RoPE, and Softmax in the attention stage. The second stage of RMSNorm, which multiplies hidden states by the normalization scale, is pipelined with RoPE. This multiplication can be moved to the first stage of RMSNorm, allowing RoPE to reuse the same set of multipliers, resolving the resource conflict between them. The remaining conflict occurs between RMSNorm and Softmax, when the k projection for the next attention group is generated early and normalized while the current softmax computation is still in progress. Simply caching the k projection output (128-element) until the softmax completes eliminates this conflict. Note that the SQRT and the FP32 IPs are exclusively used in certain modules and cannot benefit from resource sharing.

Optimizations of the FP16 Multiplier: While inter-module IP sharing greatly reduces overall resource usage, FP16 multipliers still consume multiple DSPs. ***Eliminating even a single DSP can significantly impact scalability when instantiating multiple token processors.*** An FP16 multiplication consists of an 11-bit mantissa multiplication and a 5-bit exponent addition. We observe that operand packing and DDR techniques can be applied to these operations as well shown in Fig.7, effectively halving the DSPs compared with the IP implementation. We assume two sets of FP16 inputs (x, w) and (y, z) need to be multiplied to yield xw and yz. We place the extended 11-bit mantissa of x and y on the A and D ports of the DSP, respectively, and once again exploit the INMODE[1][2] gating logic inside the DSP (2) to gate between x and y $\{(g?x : 0) + (g?0 : y)\}$, thereby eliminating the need for an additional LUT gearbox to perform the DDR trick. The operands w and z however—unless they share the same value—must be fed through an LUT gearbox and placed on the B port. Directly applying the DDR technique to the 11-bit mantissa multiplication already cuts DSP usage in half. However, we observe that further optimizations are possible by fully exploiting the DSP port width: ***notably, the addition of the FP16 exponents can also be absorbed within the DSP.*** We place the low 4 bits of the exponents x and y on A[25:22] and D[25:22], respectively, and set a single bit on B[16]. In this way, the exponents of x or y naturally appear at [41:38] of the mantissa-multiplication result. Next, we place the low 4 bits of the exponents w or z on C[41:38] and set the rounding factor RND[38] to 1 (6). By leveraging the post-adder, the operations $x + w + 1$ or $y+z+1$ will then emerge at P[42:28]. However, setting a bit on B[16] perturbs the mantissa multiplication. To correct this, we concatenate a compensation term at C[21:16] and set the rounding factor RND[16] to 1 (6). The compensation term is the bitwise reverse of the low 6 bits of the mantissa of x or y. Both the bitwise reverse and the x/y selection multiplexer form a level=1 logic that can be implemented using only 3 LUTs—consuming fewer resources than instantiating two independent 4-bit adders which require carry propagation. In this way, a pair of low 4-bit exponent additions and the 11-bit mantissa multiplications can be absorbed into a single DSP with minimal LUT overhead. Note that conventionally, exponent addition requires subtracting the bias of 15. The extra +1 operation, together with the exclusive use of the most significant bit of the exponent, are to simplify the subsequant overflow/underflow detection and normalization process. We implement these logics in the slow clock domain using LUTs, achieving identical behavior to the vendor's IP while consuming fewer LUTs and cutting DSPs in half. Our proposed optimizations on FP16 multipliers are different from the FlightVGM's methods[38] since they target DSP58.

Table 4: Comparison of different scalar engine designs

Impl.	LUT	FF	BRAM	URAM	DSP
[31]	29K	40K	6.5	3	24
[29]	5.8K	6.8K	11	0	29
Ours	5.9K	6.4K	23	0	7

Table.4 compares the resource consumption between existing scalar engine designs. We manage to consume only 7 DSPs. Note that we implement the FP16 adder entirely with LUTs, as a DSP-based design offers little LUT savings. For the EXP function, we adopt a look-up table approach to eliminate DSP usage. The higher BRAM usage results from supporting a 16K context length, whereas the two reference accelerators support only 1K and 4K, respectively.

6 Experiments

6.1 Design Choices of the Customized Platform

Our custom PCB is designed to support medium-scale MoE LLMs, overcoming the limited on-board RAM of popular embedded FPGA platforms like Ultra96, KV260/KR260, ZCU104, and ZCU102. We detail our design choices of the customized platform.

FPGA Family: A heterogeneous CPU-FPGA platform is preferred for embedded deployments, which often run standalone without a host PC. The built-in PS CPU can handle tokenization and the user interface, and the integrated DDR4 controller naturally provides a high-speed memory channel without requiring an MIG.

Product Tier: To maximize cost-effectiveness, we target low-end, entry-level FPGAs. PL-side memory integration requires only three HP banks to support a 64-bit DDR4 channel. In the Ultrascale family, even the most affordable SoCs—XCZU2CG/3EG in a 784-pin package—offer sufficient capability. Note that the 2CG is slightly more expensive (Fig.8), likely due to its lower market demand.

SODIMM or Components: On the PS side, the maximum supported DDR4 capacity is 8GB, making four 16-bit 16Gb DDR4 chips the most suitable choice. This 8GB setup is already rare among FPGA evaluation boards (usually less than 4GB). On the PL side, a DIMM interface provides higher capacity with a slight speed trade-off depending on memory rank. Support for 8/16/32GB enables scalability for larger, sparser MoE LLMs (e.g., Qwen3-80B-A3B[44]).

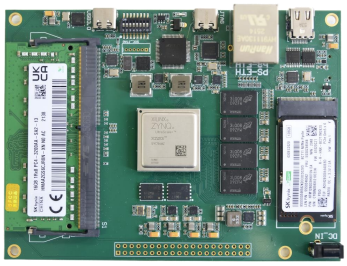
Storage: Loading model weights from storage into RAM can become a bottleneck. While SD cards or eMMC provide sufficient capacity, their bandwidth is inadequate. To address this, we leverage the PCIe 2.0 resources on the PS side and reserve an M.2 interface that supports NVMe SSDs, enabling faster weight loading.

Peripherals: Ethernet, DisplayPort, USB, and GPIO remain available on the PS side for development and integration.

The BOM for our custom platform (Fig.8) is estimated at under $150 in mass production, with DDR4 prices likely to decline soon. RAM and storage together make up nearly half the cost, ***FPGA chip itself accounts for only about 30%.*** Thus, even a cheaper ASIC replacement would have limited impact on the overall BOM.

6.2 Experiments Setup

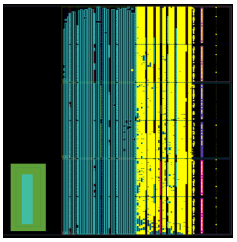
Hummingbird+ is designed in SpinalHDL[1], verified with Verilator and Vivado-sim, integrated, synthesized, and implemented in Vivado 2022.2. The model weight is split evenly between the PS and PL RAM. The PS's 8GB RAM can hold half of Qwen3-30B-A3B's 14.52GB parameters and 0.77GB *kv* cache. A bare-metal setup reserves all memory for weights and *kv* cache. This symmetric split offers the best inference speed by enabling parallel data fetches.



Components	Price ($)
XCZU3EG-SFVC784-2I[1]	45
XCZU2CG-SFVC784-2I[1]	55
8GB DDR4 Components	20
16GB DDR4 SODIMM	33
128GB SSD	17
Others	10
PCB/SMT[2]	25
Total	**150/160**

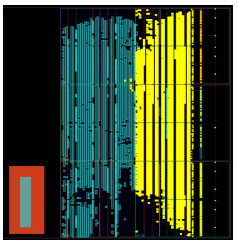
[1] The price reflects the actual purchase cost from our supplier not the public online prices from Mouser or Digi-Key. [2] Single-run PCB prototyping is expensive, this cost is an estimated average assuming a production batch of 100 pcs.

Figure 8: Bill of materials of customized platform.



XCZU3EG
Num of token processors=4
Power=7.33W (Vivado Report)

Module	LUT	FF	BRAM	DSP
global_ctrl	3496	5987	0	0
mem_ctrl	1212	4456	15.5	0
wkv_process	912	2231	0	0
engine (2×)	1895	4626	0	280
spu (4×)	23425	24353	92	28
actBuf (4×)	1368	696	12	0
Accelerator	35480	48782	119.5	308
MIG	14969	20274	25.5	3
Total	50449	69056	145	**311**
Available	70560	141120	216	360
Utilization(%)	71.50	48.93	67.13	86.39



XCZU2CG
Num of token processors=2
Power=5.77W (Vivado Report)

Module	LUT	FF	BRAM	DSP
global_ctrl	3471	5836	0	0
mem_ctrl	1202	4456	15.5	0
wkv_process	912	2231	0	0
engine (1×)	943	2210	0	140
spu (2×)	11826	12696	46	14
actBuf (2×)	669	348	6	0
Accelerator	20115	28940	67.5	154
MIG	14953	20260	25.5	3
Total	35068	49200	93	**157**
Available	47232	94464	150	240
Utilization(%)	74.25	52.08	62.00	65.42

Figure 9: Resource consumption and layout on XCZU2CG/EG.

6.3 Resource Consumption

We deploy Hummingbird+ on both XCZU2CG/3EG with different token parallelism according to the on-chip resources. The XCZU2CG (47K LUT, 240 DSP) can host 2 token processors, while the XCZU3EG (70K LUT, 360 DSP) can host 4. Each processor requires one scalar engine (7 DSP) and access to a GEMV engine. A single GEMV engine consumes 140 DSP and operates at double data rate (parallelism M=2), ***so it time-multiplexes to provide 2 virtual single-token GEMV instances.*** Hence we instantiate: (1) XCZU2CG: 2 token processors ⇒ 2 scalar engines and 1 GEMV engine. (2) XCZU3EG: 4 token processors ⇒ 4 scalar engines and 2 GEMV engines. The resource consumption breakdown of both designs are shown in Fig. 9. ***Hummingbird+ targets the lowest-end product tier among existing LLM accelerators, and our design pushes the resource utilization of these two chips to their limits.*** In Fig.9, the yellow region highlights the logic resources for MIG.

6.4 Performance Comparison

To ensure authenticity, we report only raw measurements rather than normalized metrics, ***even when comparing against devices that are significantly more expensive and resource-rich.***

6.4.1 Performance with embedded GPU and CPU. Qwen3-30B-A3B targets personal computers, so we select DDR4-based CPUs and the Jetson AGX Orin as baselines (Fig.10, Table.6). Power consumption is measured with an external power meter. We use `llama.cpp`[17] framework with the Q4_0_GGUF format on both CPU and GPU. For decode speed, we fix the prefill sequence length at 32 and vary the output length from 32 to 16384. XCZU2CG shows lower speed than XCZU3EG at longer contexts, since it integrates only half as many token processors, limiting its GQA performance. Both platform achieves nearly 2× higher decoding throughput than a DDR4-based CPU under the same bandwidth. Compared to Jetson AGX Orin, we sustain over half its decoding speed while using only 17% of its bandwidth, achieving 7× higher token-per-dollar efficiency and

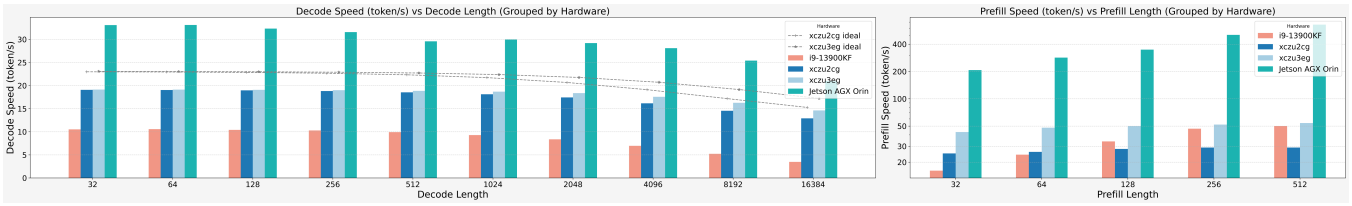


Figure 10: Decode and prefill speed comparision with commercial CPU and embedded GPU.

Table 5: Comparison of FPGA-based LLM Accelerators

Research	Platform	Model	Acc.Stage	Compression	LUTs	DSPs	BW(GB/s)	Power(W)	D.token/s	P.token/s
FlightLLM[62]	U280	Llama2-7B	P&D	SparseW8A8	574K	6345	460	45	55	2-4×
EdgeLLM[23]	VCU128	ChatGLM-6B	P&D	W4A16	967K	4563	460	56.8	86	2×
AccLLM[35]	U280	Llama2-7B	P&D	W2KV4A8	420K	4497	460	33	164	2-4×
Chen et al.[8]	U280	GPT-2	P&D	W4A8	569K	1780	460	29.5	204	-
ChatOPU[66]	U200	OPT-350M	P&D	SparseW16	755K	1091	19.2	-	44	-
LoopLynx[67]	U50	GPT2-345M	D	W8	312K	1131	201	39.4	260	1×
LightMamba[57]	VCK190	Mamba2-2.7B	D	W4	107K	228	12	3.2	7.2	1×
LPU[42]	U55C	OPT-6.7B	D	W16A16	605K	2482	460	-	30	1×
Li et al.[31]	KV260	Llama2-7B	D	W4A16	78K	291	19.2	6.57	4.9	1×
Hummingbird[29]	KV260	Llama3-8B	D	W4A24	26K	179	19.2	3.81	4.8	1×
TeLLMe[43]	KV260	BitNet	P&D	W1.58	108K	356	19.2	7	9	116
Xiang et al.[59]	KV260	Qwen-0.5B	P&D	W4	96K	384	19.2	-	5.1	-
Ours	**ZU3EG**	**Qwen3-30B-A3B**	**P&D**	W4KV8A12	50K	311	34	7.33 (~13)	18	50 (Peak 4×)

Table 6: Specifications of baseline platforms

	RAM	Capacity	Bandwidth	Cost	TOPS	Power
AGXOrin	LPDDR5	64GB	204GB/s	$1,999	275	~40W
i9-13900KF	DDR4 2ch	32GB	34GB/s	$1,000	~1	~180W
XCZU3EG	DDR4 2ch	24GB	34GB/s	$150	0.5	~13W

1.7× higher power efficiency. For prefill speed, we vary the sequence length from 32 to 512. Our XCZU3EG outperforms CPUs in all prefill lengths, but Orin achieves much higher prefill throughput due to its greater compute resources. We can also outperform embedded NPU, as [5] reports nearly 10 token/s speed on a 3B LLM on RK3588—one of the most cost-efficient edge AI devices.

6.4.2 Performance with SOTA accelerator. As shown in Table.5, Hummingbird+ delivers the best decoding performance among embedded FPGA-based accelerators, while running the largest LLM on the smallest device with prefill acceleration. We also achieve comparable performance compared to many cloud FPGA-based research. The prefill performance of prior work is difficult to derive from the normalized latency reported, but relative speedup over decode can be estimated from DSP usage. Our accelerator reserves 4× compute for prefill acceleration—same as FlightLLM. However, due to MoE workload irregularity and activation offloading/reloading, the actual prefill throughput is expected to be around 50 token/s.

7 Limitations, Discussions, Future Perspectives

Scalability. Our customized platform can further scale to 8+32=40GB RAM, sufficient for the latest Qwen3-Next-80B-A3B in INT4. More broadly, FPGA scalability stems from its flexible HP banks, which can serve as a **glue chip** to integrate large-capacity RAM for MoE LLMs. For example, the KU040[6] can interface with a 160-bit DDR4 (64+64+32) supporting up to 72GB. With SerDes in advanced FPGA, scaling to even multi-FPGA systems is also feasible[40].

Adaptability. The key to enabling LLMs on low-cost FPGAs is a **hand-crafted RTL design** with aggressive optimizations. Overlay-based DPUs[27], HLS[9], and compiler-driven[61] methods cannot reach this level of fine-grained efficiency, forcing the use of more expensive FPGAs to accommodate redundant logic. Fortunately, LLM architectures are relatively stable, making model adaptation easier than in CNNs or ViTs, a promising opportunity for automated tools[50] to generate optimized RTL for LLM accelerators.

Competition with ASICs. We show that the cost of RAM and storage needed for MoE-based LLM already exceeds that of the FPGA itself. Although an ASIC could offer higher compute FLOPs, the decoding stage remains memory-bound. If FPGA proves feasible for edge LLM deployment and ASICs provide no strong benefit (when chip cost and FLOPs are no longer the bottleneck), ASIC may be less attractive in this fast-changing domain given the MoE-LLM only gaining popularity recently in the rapid evolution of LLMs.

8 Conclusions

In this work, we present Hummingbird+, a low-cost FPGA platform and accelerator that enables deployment of a 30B MoE-based LLM. Our customized system delivers 18 token/s decoding and 50 token/s prefill throughput with an estimated $150 BOM cost. For the first time, we have shown that FPGA-based LLM deployment can be advanced from a research prototype to an edge-ready product.

Acknowledgments

This study was funded by the National Natural Science Foundation of China (32571284), the State Key Laboratory of Brain Cognition and Brain-inspired Intelligence Technology (JS202401), and the Beijing Municipal Bureau of Economy and Information Technology.

References

[1] [n. d.]. SpinalHDL: Scala based HDL. https://github.com/SpinalHDL/SpinalHDL

[2] Megha Agarwal, Asfandyar Qureshi, Linden Li Nikhil Sardana, Julian Quevedo, and Daya Khudia. 2023. Llm inference performance engineering: Best practices.

[3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).

[4] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245* (2023).

[5] airockchip. 2025. Benchmark — RKNN-LLM. https://github.com/airockchip/rknn-llm/blob/main/benchmark.md. Accessed: 2025-10-02.

[6] AMD Inc. 2025. *UltraScale+ Device Packaging and Pinouts: Product Specification User Guide (UG575).* Technical Report UG575. AMD. https://docs.amd.com/v/u/en-US/ug575-ultrascale-pkg-pinout Accessed: 2025-10-02.

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[8] Hongzheng Chen, Jiahao Zhang, Yixiao Du, Shaojie Xiang, Zichao Yue, Niansong Zhang, Yaohui Cai, and Zhiru Zhang. 2024. Understanding the potential of fpga-based spatial acceleration for large language model inference. *ACM Transactions on Reconfigurable Technology and Systems* 18, 1 (2024), 1–29.

[9] Hongzheng Chen, Niansong Zhang, Shaojie Xiang, Zhichen Zeng, Mengjia Dai, and Zhiru Zhang. 2024. Allo: A programming model for composable accelerator design. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 593–620.

[10] Yuze Chi, Weikang Qiao, Atefeh Sohrabizadeh, Jie Wang, and Jason Cong. 2022. Democratizing domain-specific computing. *Commun. ACM* 66, 1 (2022), 74–85.

[11] Eric Chung, Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Adrian Caulfield, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, et al. 2018. Serving dnns in real time at datacenter scale with project brainwave. *iEEE Micro* 38, 2 (2018), 8–20.

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers).* 4171–4186.

[13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

[14] Zelin Du, Wei Zhang, Zimeng Zhou, Zili Shao, and Lei Ju. 2023. Accelerating dnn inference with heterogeneous multi-dpu engines. In *2023 60th ACM/IEEE Design Automation Conference (DAC).* IEEE, 1–6.

[15] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).

[16] Yao Fu, Ephrem Wu, and Ashish Sirasao. 2017. 8-bit dot-product acceleration. *Xilinx Inc.: San Jose, CA, USA* (2017), 20.

[17] Georgi Gerganov and contributors. 2025. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp. Accessed: 2025-10-02.

[18] Kaiyuan Guo, Lingzhi Sui, Jiantao Qiu, Jincheng Yu, Junbin Wang, Song Yao, Song Han, Yu Wang, and Huazhong Yang. 2017. Angel-eye: A complete design flow for mapping CNN onto embedded FPGA. *IEEE transactions on computer-aided design of integrated circuits and systems* 37, 1 (2017), 35–47.

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition.* 770–778.

[20] Zicheng He, Tiandong Zhao, Siyuan Miao, Chen Wu, and Lei He. 2024. An FPGA-based efficient streaming vector processing engine for transformer-based models. In *2024 2nd International Symposium of Electronics Design Automation (ISEDA).* IEEE, 722–727.

[21] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. 2020. Query-key normalization for transformers. *arXiv preprint arXiv:2010.04245* (2020).

[22] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. 2024. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. *Proceedings of Machine Learning and Systems* 6 (2024), 148–161.

[23] Mingqiang Huang, Ao Shen, Kai Li, Haoxiang Peng, Boyu Li, Yupeng Su, and Hao Yu. 2025. Edgellm: A highly efficient cpu-fpga heterogeneous edge accelerator for large language models. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2025).

[24] Zhirui Huang, Rui Ma, Shijie Cao, Ran Shu, Ian Wang, Ting Cao, Chixiao Chen, and Yongqiang Xiong. 2025. TENET: An Efficient Sparsity-Aware LUT-Centric Architecture for Ternary LLM Inference On Edge. *arXiv preprint arXiv:2509.13765* (2025).

[25] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).

[26] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture.* 1–14.

[27] Vinod Kathail. 2020. Xilinx vitis unified software platform. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays.* 173–174.

[28] Cong Li, Yihan Yin, Xintong Wu, Jingchen Zhu, Zhutianya Gao, Dimin Niu, Qiang Wu, Xin Si, Yuan Xie, Chen Zhang, et al. 2025. H2-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture.* 194–210.

[29] Jindong Li, Tenglong Li, Ruiqi Chen, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2025. Hummingbird: A Smaller and Faster Large Language Model Accelerator on Embedded FPGA. *Proceedings of the 44rd IEEE/ACM International Conference on Computer-Aided Design* (2025).

[30] Jindong Li, Tenglong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2024. Revealing untapped dsp optimization potentials for fpga-based systolic matrix engines. In *2024 34th International Conference on Field-Programmable Logic and Applications (FPL).* IEEE, 197–203.

[31] Jindong Li, Tenglong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2025. Pushing up to the limit of memory bandwidth and capacity utilization for efficient llm decoding on embedded fpga. In *2025 Design, Automation & Test in Europe Conference (DATE).* IEEE, 1–7.

[32] Jindong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2023. Firefly: A high-throughput hardware accelerator for spiking neural networks with efficient dsp and memory optimization. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 31, 8 (2023), 1178–1191.

[33] Jindong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2024. Firefly v2: Advancing hardware support for high-performance spiking neural network with a spatiotemporal fpga accelerator. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 9 (2024), 2647–2660.

[34] Tenglong Li, Jindong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. 2024. Firefly-s: Exploiting dual-side sparsity for spiking neural networks acceleration with reconfigurable spatial architecture. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2024).

[35] Yanbiao Liang, Huihong Shi, Haikuo Shao, and Zhongfeng Wang. 2025. AccLLM: Accelerating Long-Context LLM Inference Via Algorithm-Hardware Co-Design. *arXiv preprint arXiv:2505.03745* (2025).

[36] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).

[37] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[38] Jun Liu, Shulin Zeng, Li Ding, Widyadewi Soedarmadji, Hao Zhou, Zehao Wang, Jinhao Li, Jintao Li, Yadong Dai, Kairui Wen, et al. 2025. Flightvgm: Efficient video generation model inference with online sparsification and hybrid precision on fpgas. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays.* 2–13.

[39] Shaoqiang Lu, Xuliang Yu, Tiandong Zhao, Siyuan Miao, Xinsong Sheng, Chen Wu, Liang Zhao, Ting-Jung Lin, and Lei He. 2025. MambaOPU: An FPGA Overlay Processor for State-space-duality-based Mamba Models. In *2025 62nd ACM/IEEE Design Automation Conference (DAC).* IEEE, 1–7.

[40] Wenheng Ma, Xinhao Yang, Shulin Zeng, Tengxuan Liu, Libo Shen, Hongyi Wang, Shiyao Li, Jiewen Wang, Yuhan Zhang, Hao Guo, et al. 2025. FMC-LLM: Enabling FPGAs for Efficient Batched Decoding of 70B+ LLMs with a Memory-Centric Streaming Architecture. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays.* 55–55.

[41] Maxim Milakov and Natalia Gimelshein. 2018. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867* (2018).

[42] Seungjae Moon, Jung-Hoon Kim, Junsoo Kim, Seongmin Hong, Junseo Cha, Minsu Kim, Sukbin Lim, Gyubin Choi, Dongjin Seo, Jongho Kim, et al. 2024. Lpu: A latency-optimized and highly scalable processor for large language model inference. *IEEE Micro* (2024).

[43] Ye Qiao, Zhiheng Chen, Yifan Zhang, Yian Wang, and Sitao Huang. 2025. TeLLMe: An Energy-Efficient Ternary LLM Accelerator for Prefilling and Decoding on Edge FPGAs. *arXiv preprint arXiv:2504.16266* (2025).

[44] Qwen Team. 2025. Qwen3-Next-80B-A3B-Instruct. https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct. Accessed: 2025-10-02.

[45] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*

1, 8 (2019), 9.

[46] Ltd. Rockchip Electronics Co. 2022. *RK3588 Brief Datasheet.* https://www.rock-chips.com/uploads/pdf/2022.8.26/192/RK3588%20Brief%20Datasheet.pdf

[47] Ananda Samajdar, Tushar Garg, Tushar Krishna, and Nachiket Kapre. 2019. Scaling the cascades: Interconnect-aware FPGA implementation of machine learning problems. In *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 342–349.

[48] Jan Sommer, M Akif Özkan, Oliver Keszocze, and Jürgen Teich. 2022. Dsp-packing: Squeezing low-precision arithmetic into fpga dsp blocks. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 160–166.

[49] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.

[50] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.

[51] Ajay Tirumala and Raymond Wong. 2024. Nvidia blackwell platform: Advancing generative ai and accelerated computing. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 1–33.

[52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[53] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[54] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[55] Xuanzheng Wang, Shuo Miao, Peng Qu, and Youhui Zhang. 2024. DTrans: A Dataflow-transformation FPGA Accelerator with Nonlinear-operators fusion aiming for the Generative Model. In *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 339–345.

[56] Zhican Wang, Hongxiang Fan, Haroon Waris, Gang Wang, Zhenyu Li, Jianfei Jiang, Yanan Sun, and Guanghui He. 2025. VEDA: Efficient LLM Generation Through Voting-based KV Cache Eviction and Dataflow-flexible Accelerator. *arXiv preprint arXiv:2507.00797* (2025).

[57] Renjie Wei, Songqiang Xu, Linfeng Zhong, Zebin Yang, Qingyu Guo, Yuan Wang, Runsheng Wang, and Meng Li. 2025. Lightmamba: Efficient mamba acceleration on fpga with quantization and hardware co-design. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.

[58] Ephrem Wu, Xiaoqian Zhang, David Berman, and Inkeun Cho. 2017. A high-throughput reconfigurable processing array for neural networks. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 1–4.

[59] Maoyang Xiang, Ramesh Fernando, and Bo Wang. 2025. On-Device Qwen2. 5: Efficient LLM Inference with Model Compression and Hardware Acceleration. *arXiv preprint arXiv:2504.17376* (2025).

[60] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[61] Hanchen Ye and Deming Chen. 2025. StreamTensor: Make Tensors Stream in Dataflow Accelerators for LLMs. *arXiv preprint arXiv:2509.13694* (2025).

[62] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, et al. 2024. Flightllm: Efficient large language model inference with a complete mapping flow on fpgas. In *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 223–234.

[63] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems* 32 (2019).

[64] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 161–170.

[65] Jingwei Zhang, Meng Zhang, Xinye Cao, and Guoqing Li. 2023. Uint-packing: Multiply your dnn accelerator performance via unsigned integer dsp packing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[66] Tiandong Zhao, Shaoqiang Lu, Chen Wu, and Lei He. 2024. ChatOPU: An FPGA-based Overlay Processor for Large Language Models with Unstructured Sparsity. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

[67] Jianing Zheng and Gang Chen. 2025. LoopLynx: A Scalable Dataflow Architecture for Efficient LLM Inference. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.